Vantara Customer Behavior Prediction Platform

Final report: churn, lifetime value and purchase behavior on Online Retail II

Student project built for the Internmo traineeship. The data is the public UCI Online Retail II dataset, a real UK retailer, used here as a stand-in for Vantara's data. The system is a working prototype for learning and demonstration, not a production tool.

1. Summary and honest results

Metric	Target	Result	Status
Churn ROC-AUC (test)	>= 0.8	0.786	NOT MET
Churn recall, churned class (test)	>= 0.7	0.765	MET
CLV R2, next-90-day spend (test, raw scale)	>= 0.6	0.415	NOT MET
API p95, single prediction	< 400 ms	6.6 ms	MET (measured)
Dashboard load, segment view	< 3 s	1.7-2.2 s warm	MET warm, see 9

Two model-quality targets (churn ROC-AUC and CLV R2) are missed and are discussed in section 9. The production churn model is **xgboost**, chosen by validation AUC among the gradient-boosted candidates, as the brief recommends. Validation AUC: xgboost 0.811, Random Forest 0.808; test AUC 0.786 vs 0.790. The gap is within split noise (see section 4). Every choice was made on training and validation data. Test metrics are computed once per model and logged beside validation metrics, but never drive a decision.

2. Data and cleaning

Source: UCI Machine Learning Repository, Online Retail II (id 502, CC BY 4.0, Chen 2012): 0 invoice lines from 01 Dec 2009 to 09 Dec 2011. Both sheets are loaded, standardized and ordered by time before any cleaning. Each cleaning rule is counted:

Rule	Count
Lines in	0
Missing Customer ID (excluded from customer-level modelling)	0
Zero or negative price (adjustments, dropped)	0
Non-product stock codes (postage, bank charges, manual, ...)	0
Exact duplicate lines	0
Quantity above IQR bound (0, fitted on training customers) kept as bulk buys	0
Price above IQR bound (0.00, fitted on training customers) kept	0
Quantity above bound and >= 1,000 (data-entry errors, dropped)	0
Price above bound and >= 1,000 (data-entry errors, dropped)	0
Lines out	0
Customers with an ID	0

Returns (negative quantity, invoices starting with C) are kept and used as a signal (return rate), not dropped. Product descriptions are standardized with a lookup keyed on stock code. The IQR bounds, the description lookup, product median prices and the engagement percentile reference are fitted on training customers' pre-cutoff rows only (customers are split before any statistic is fitted) and persisted in models_artifacts/reference_stats.joblib. Automated checks (schema, date range, nulls, price, ordering) run before feature engineering and stop the pipeline if violated.

3. Features, target definitions and leakage control

All features use only lines with invoice date before the cutoff **2011-09-09**. The churn label is 1 when the customer has no purchase in the 90 days from the cutoff (window ends before the last data date, so the label window is complete). The CLV target is each customer's net spend in that same window (returns subtract, negative totals are floored at 0). The modelled population is the 5,248 customers with at least one purchase before the cutoff, and the churn rate is 56.6%: in this window churn is the *majority* class, so imbalance handling is mild; class weights are still used.

Features: recency, frequency, monetary total and average, tenure, basket size, orders and spend in the last 30/90/180 days, purchase frequency trend (slope of monthly orders over 12 months), inter-purchase gap mean and variance, seasonal concentration, product category affinity vector (keyword categories, because the source has no category field), return rate, discount sensitivity (share of lines priced under 90% of the product median), country one-hot, historical CLV and a composite engagement score. Historical CLV is net of returns and cancellations, so a few customers have negative values (for example customer 12346); these are real records and are kept, and the dashboard flags them. Unit tests change every row on or after the cutoff and assert that all features stay identical.

4. Churn models

Six classical models (logistic regression, decision tree, random forest, XGBoost, LightGBM, KNN) plus a feed-forward ANN (batch norm, dropout, early stopping) and an LSTM over each customer's last 8 purchases. Split is stratified 70/15/15 with seed 42. Hyperparameters were tuned by 5-fold stratified cross-validated random search on the training split only; scale-sensitive models use a StandardScaler fitted on training data. Table is ordered by validation ROC-AUC then validation recall; test metrics are shown for reference, reported once, and never used for ordering, selection or tuning.

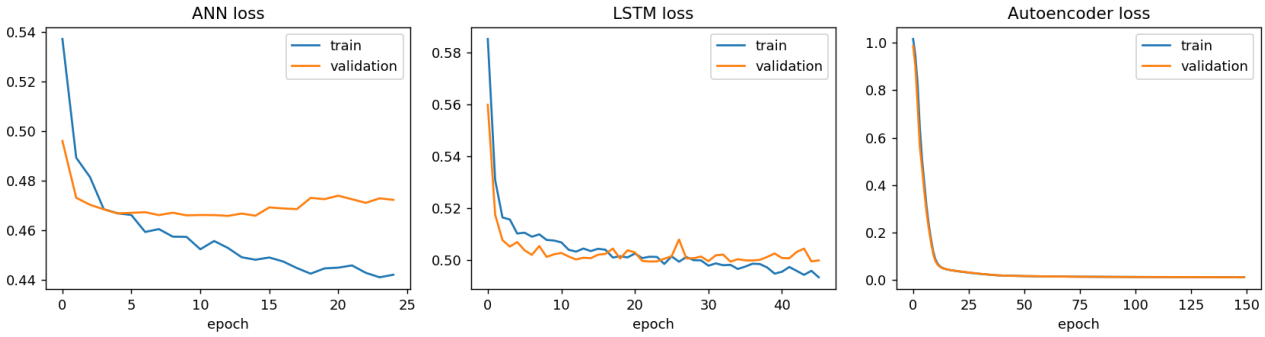
model	val_roc_auc	test_roc_auc	test_recall	test_precision	test_f1	test_accuracy	test_tn	test_fp	test_fn	test_tp	val_recall
xgboost	0.811	0.786	0.765	0.765	0.765	0.734	237.000	105.000	105.000	341.000	0.778
random_forest	0.808	0.790	0.783	0.760	0.771	0.737	232.000	110.000	97.000	349.000	0.812
ann	0.804	0.787	0.758	0.763	0.760	0.730	237.000	105.000	108.000	338.000	0.769
logistic_regression	0.801	0.790	0.767	0.758	0.763	0.730	233.000	109.000	104.000	342.000	0.769
lightgbm	0.801	0.777	0.760	0.745	0.752	0.717	226.000	116.000	107.000	339.000	0.780
knn	0.790	0.778	0.785	0.737	0.760	0.720	217.000	125.000	96.000	350.000	0.821
decision_tree	0.777	0.753	0.783	0.743	0.762	0.723	221.000	121.000	97.000	349.000	0.809
lstm	0.763	0.740	0.724	0.726	0.725	0.689	220.000	122.000	123.000	323.000	0.767

Gradient-boosted trees are the production recommendation. The ANN reaches a similar AUC (test 0.787) but is harder to explain; the LSTM is weaker (0.740) because with about 5,000 customers and 8 purchases per sequence it adds little beyond the engineered features. The deep models are comparative learning exercises, as the brief's risk table anticipates.

Cross-validation (5-fold stratified, training customers only), mean and standard deviation of the tuning metric:

task	model	metric	cv mean	cv std	folds
churn	logistic_regression	ROC-AUC	0.804	0.018	5
churn	decision_tree	ROC-AUC	0.780	0.015	5
churn	random_forest	ROC-AUC	0.810	0.013	5
churn	xgboost	ROC-AUC	0.797	0.011	5
churn	lightgbm	ROC-AUC	0.799	0.011	5
churn	knn	ROC-AUC	0.798	0.009	5
churn	ann	ROC-AUC	0.806	0.013	5
churn	lstm	ROC-AUC	0.759	0.014	5

task	model	metric	cv mean	cv std	folds
timing30	static_lgbm	ROC-AUC	0.778	0.023	5
timing30	lstm_sequence	ROC-AUC	0.758	0.011	5
timing_days	static_lgbm	MAE days	20.736	0.764	5
timing_days	lstm_sequence	MAE days	21.330	1.064	5
next_category	static_lgbm	accuracy	0.447	0.018	5
next_category	lstm_sequence	accuracy	0.457	0.018	5
clv	ridge	R2 (log1p)	0.338	0.025	5
clv	random_forest_reg	R2 (log1p)	0.348	0.021	5
clv	xgboost_reg	R2 (log1p)	0.327	0.030	5



Training vs validation loss for the ANN, LSTM and autoencoder (early stopping restores the best validation epoch).

Next-purchase timing and category (sequence model)

The LSTM reads each customer's last 8 purchases (amount, gap in days, product category) and is compared with a static-feature LightGBM and their average, for three targets: a 30-day purchase flag (all customers), days until the next purchase and the category of the next purchase (both among the 342 test customers who buy within the 90-day horizon; 1594 used for training). Test results, reported once:

model	30-day AUC	days MAE	days RMSE	days R2 (log)	category acc.	category top-3
static_lgbm	0.781	20.6	25.5	0.159	0.471	0.825
lstm_sequence	0.740	22.0	27.3	0.097	0.503	0.816
average	0.778	21.2	26.2	0.155	0.497	0.833
train_median / majority baseline		22.2	26.3	-0.036	0.418	

Whether the LSTM adds value beyond static features is read from the table: it only counts if it beats the static model and the baseline. Categories are keyword-rule groups, because the source has no category field.

Product recommendations

An item-item co-purchase model (cosine similarity between products, fitted on training customers' purchases before the cutoff; 3,792 products bought by at least 5 training customers) scores products a customer has not bought. Offline check on 342 test-split buyers: a recommended product (top 10) appears among their purchases in the next 90 days for 34.8% of them, against 26.3% for recommending the most popular products. This is a sanity check, not evidence of campaign impact: the brief puts live A/B testing out of scope. Recommendations are served at GET /customers/{id}/recommendations and shown on the dashboard's customer view.

Anomaly detection

An autoencoder trained on six scaled spending features flags customers whose reconstruction error exceeds the 99th percentile of validation errors (threshold 0.159). On the test split it flags 0.8% of customers. Flags are

for manual review; no ground truth for fraud exists in this dataset, so no accuracy is claimed.

5. Customer lifetime value (next 90 days)

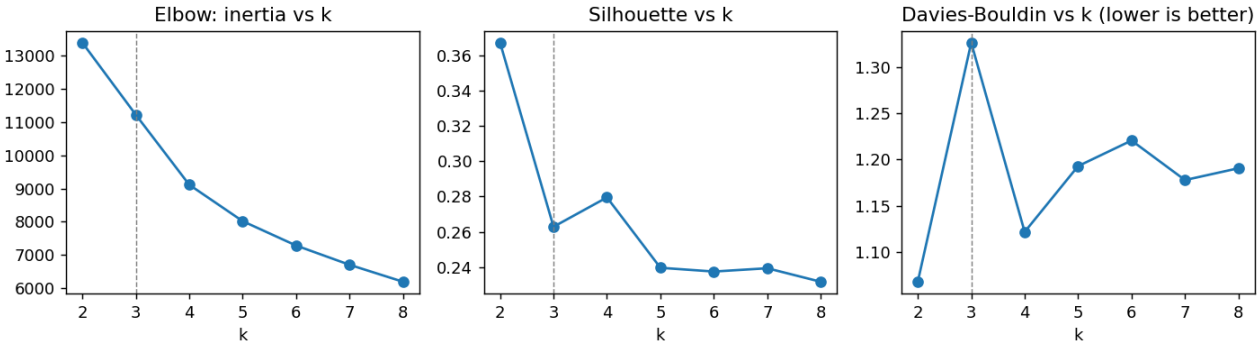
model	val R2 (raw)	test R2 (raw)	test R2 (log1p)	test MAE	test RMSE
ridge	0.770	0.415	0.326	422	2752
random_forest_reg	0.234	0.463	0.323	396	2635
xgboost_reg	0.118	0.406	0.322	416	2773

Models are trained on log1p(spend) and scored back on the raw scale, with predictions capped at the largest training target. The chosen model is **ridge** (best validation R2). The target is 90-day future spend (not a discounted-value proxy). Test R2 is 0.415 on the raw scale; Random Forest scores higher on test but lower on validation, so it was not selected (no selection on test). Ridge log-scale R2 is 0.326.

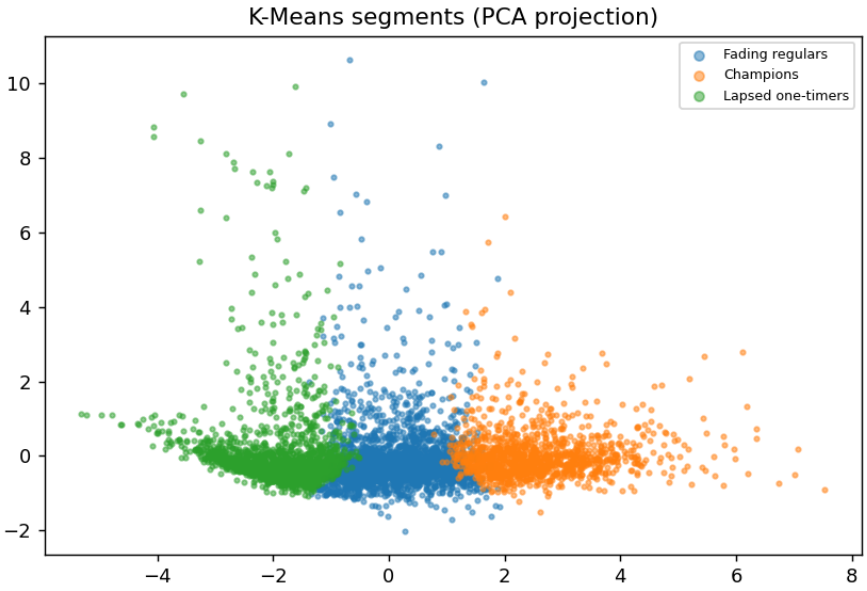
6. Segmentation

segment_name	recency_days	frequency	monetary_total	avg_basket_size	return_rate	engagement_score	churn	future_spend	customers
Fading regulars	182.7	3.63	1247.74	258.52	0.03	52.95	0.56	256.59	2180
Champions	42.96	16.02	8365.5	275.08	0.03	83.81	0.2	1590.7	1208
Lapsed one-timers	338.8	1.42	264.02	112.92	0.02	23.28	0.81	94.49	1860

K-Means was fitted for k = 2..8 on log-scaled recency, frequency, spend, basket size, return rate and engagement. The scaler, K-Means, GMM and DBSCAN are fitted on training customers only and then applied to every customer (held-out silhouette 0.264, Davies-Bouldin 1.277 at k = 3). k is read from the elbow curve (inertia), silhouette and Davies-Bouldin index (figure below). Silhouette is highest at k = 2 (0.367); k = 3 (0.263) was used because three business-readable segments are more actionable, a judgment call. Segment names are rule-based from centroid spend and recency. A Gaussian mixture with 4 components scored silhouette 0.158 (Davies-Bouldin 1.618). K-Means Davies-Bouldin at the chosen k (train) is 1.326. DBSCAN (eps 0.92) found 1 dense cluster and labelled 5.6% as noise, which says the customer base is one continuous cloud rather than separated groups, so K-Means' spherical assumption is not obviously hiding other shapes.

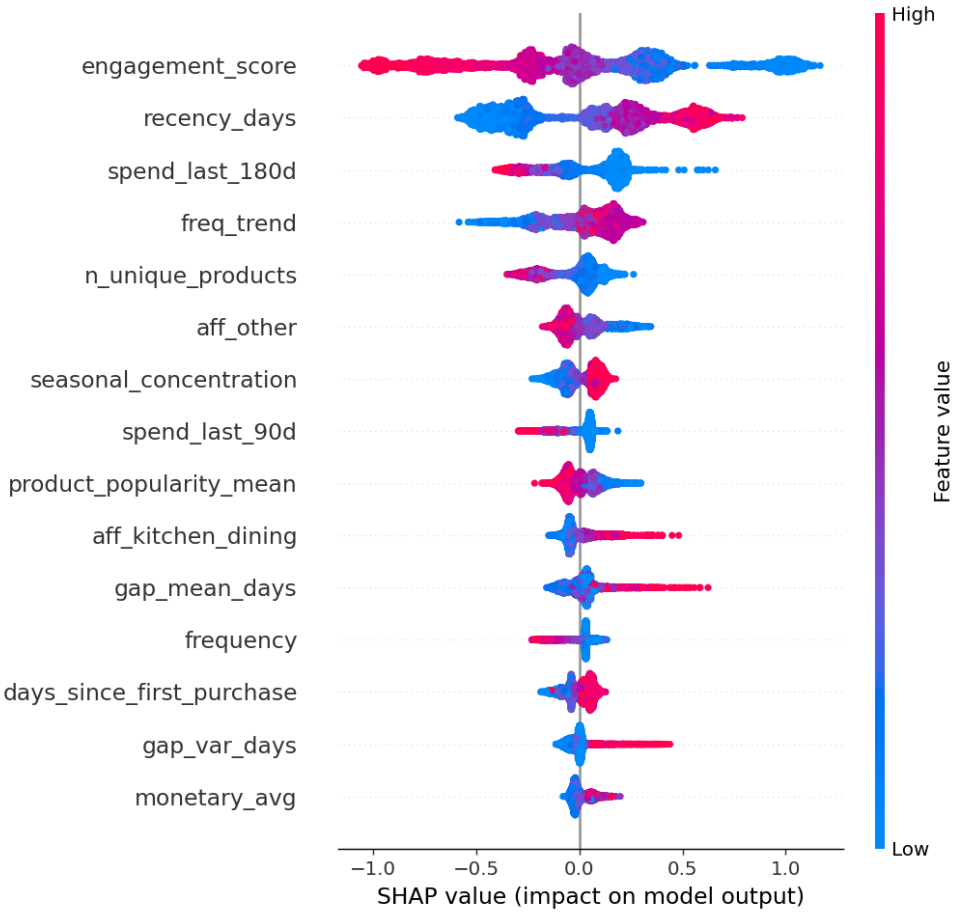


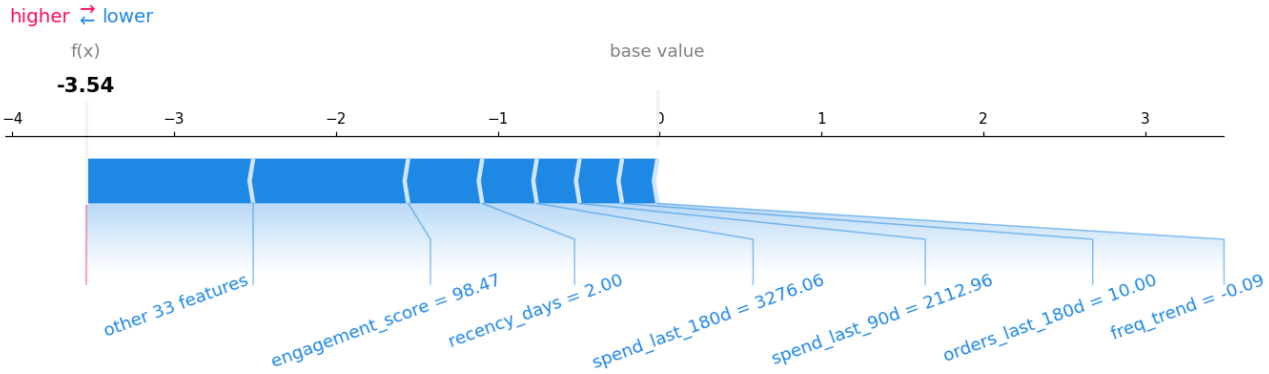
Elbow (inertia), silhouette and Davies-Bouldin against k, training customers.

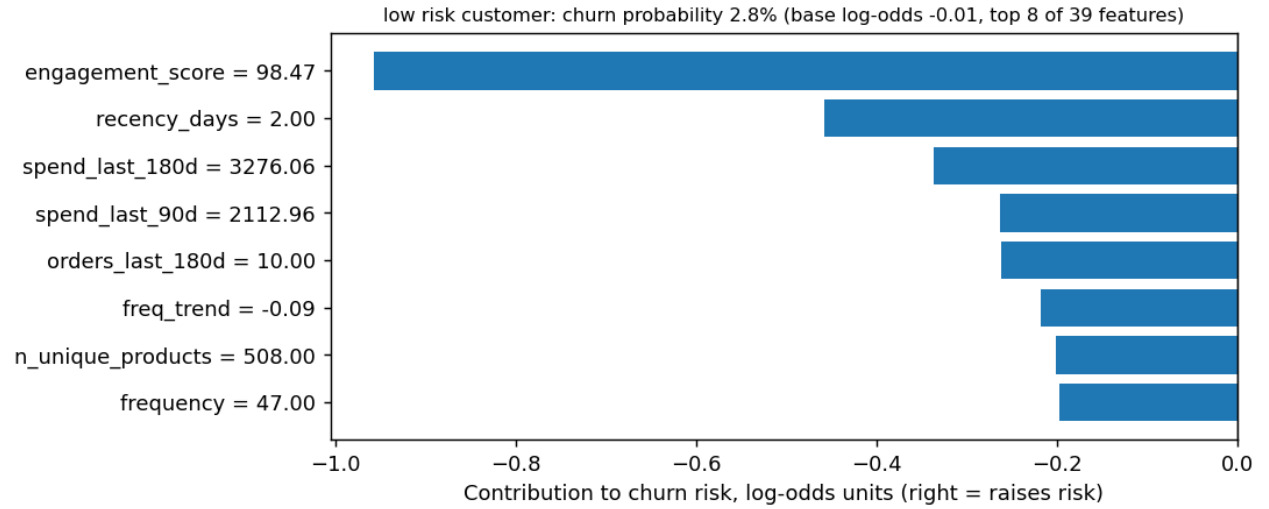


7. Explainability

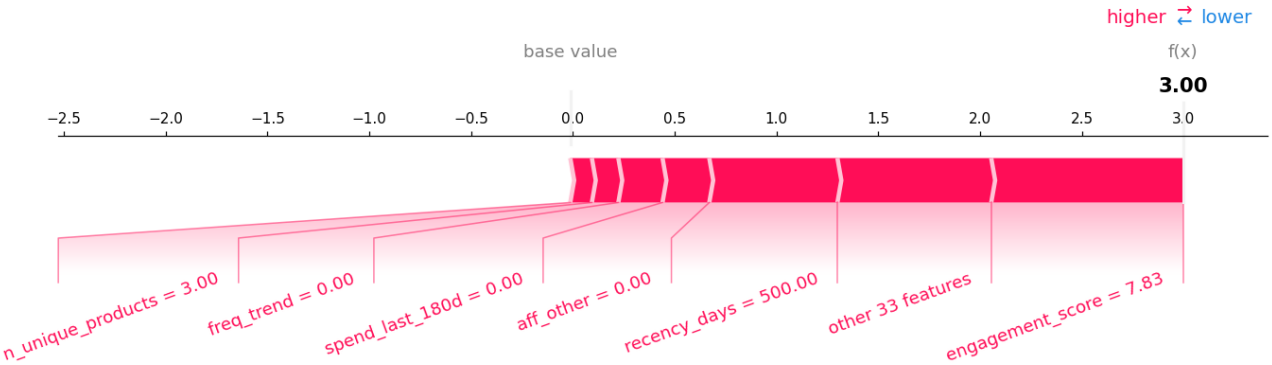
Feature contributions (native XGBoost pred_contribs, equal to tree SHAP, in log-odds units) gives the global importance below; the three most influential features are engagement_score, recency_days, spend_last_180d. The dashboard shows the same SHAP breakdown for any customer ID.

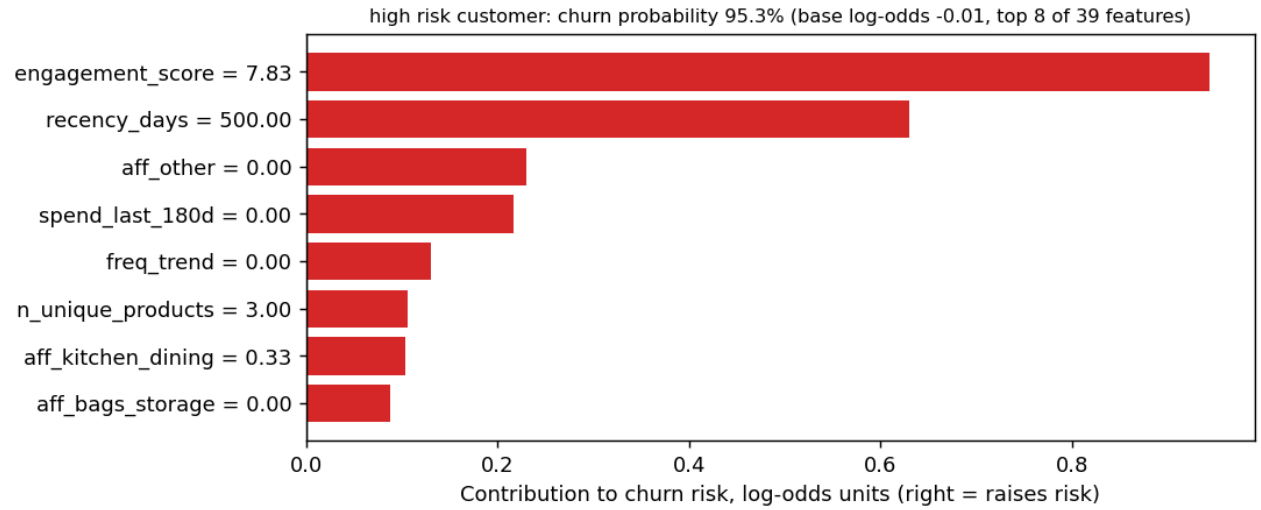




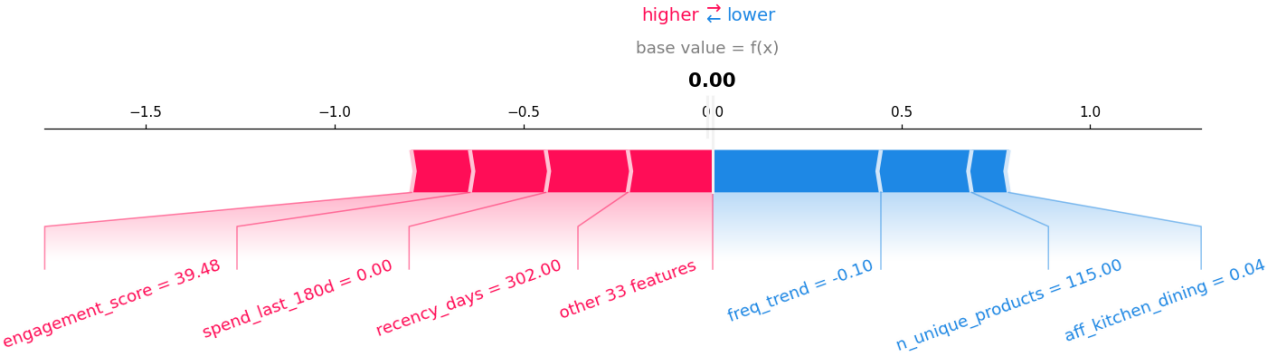


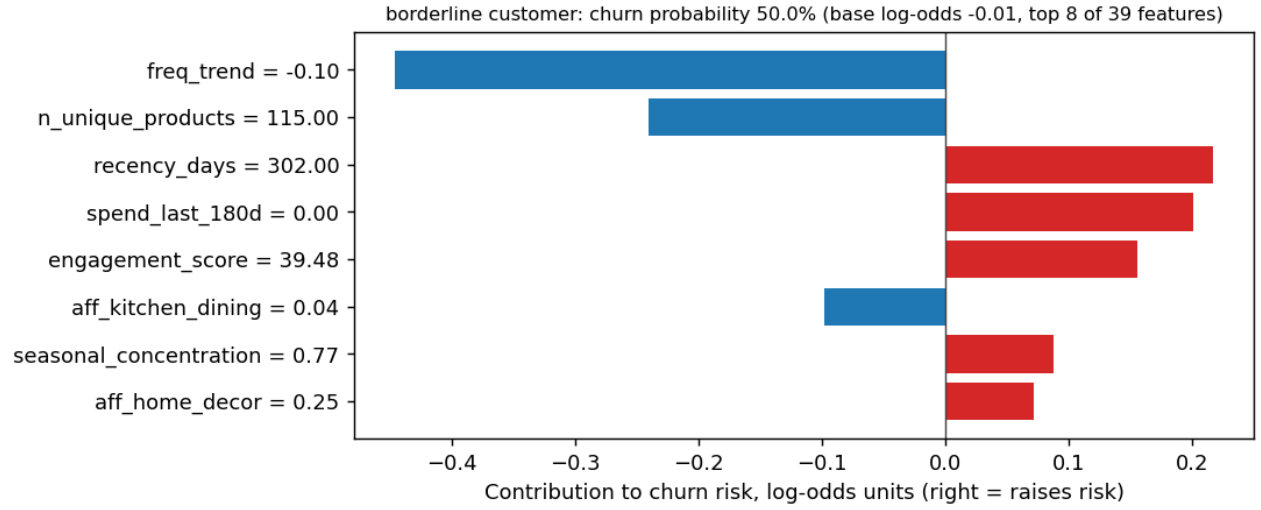
Low-risk customer 14667 (probability 2.8%). SHAP force plot (top 6 contributions, the other features summed into one term; log-odds scale) and the same contributions as a labelled bar chart. Plain-language template: Churn risk is low (3%) mainly because this customer has an engagement score of 98/100 (lowers risk); has not bought for 2 days (lowers risk); spent 3,276 in the last 180 days (lowers risk).





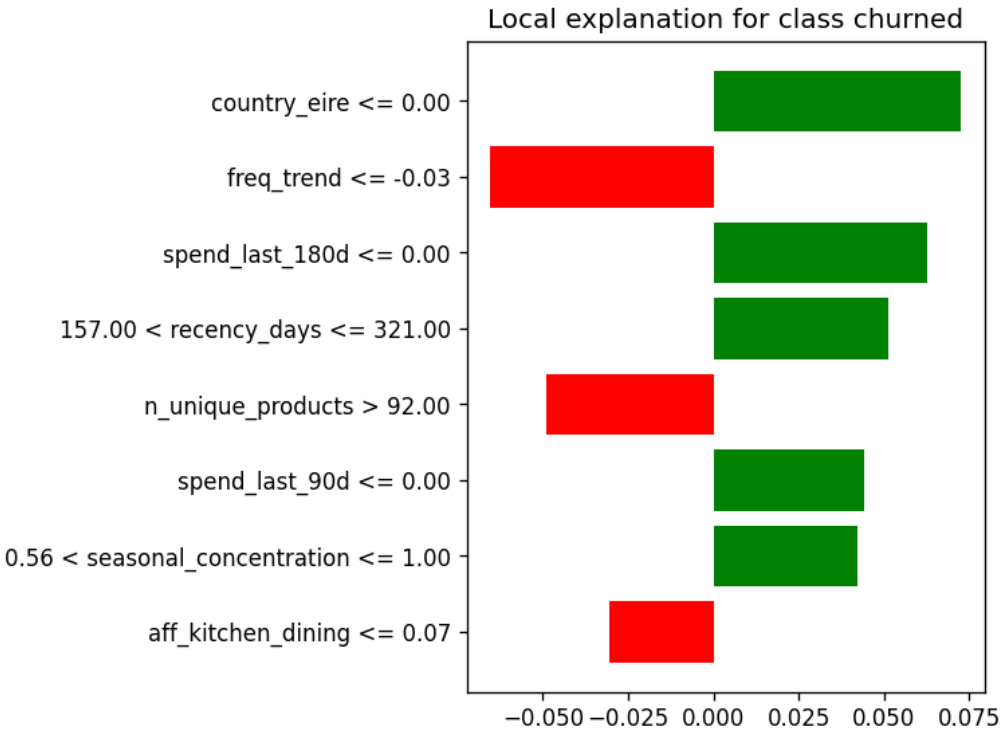
High-risk customer 14743 (probability 95.3%). SHAP force plot (top 6 contributions, the other features summed into one term; log-odds scale) and the same contributions as a labelled bar chart. Plain-language template: Churn risk is high (95%) mainly because this customer has an engagement score of 8/100 (raises risk); has not bought for 500 days (raises risk); buys 0% of items from the other category (raises risk).



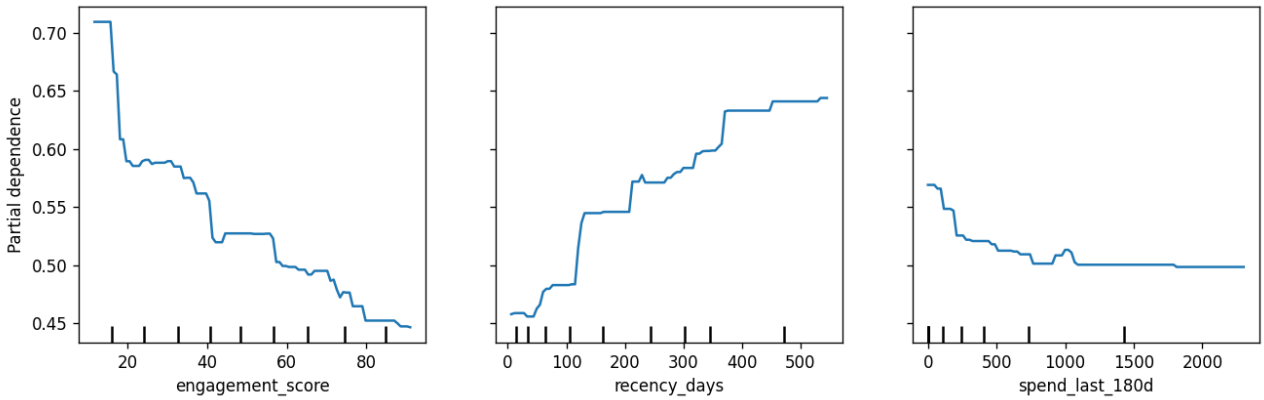


Borderline customer 15276 (probability 50.0%). SHAP force plot (top 6 contributions, the other features summed into one term; log-odds scale) and the same contributions as a labelled bar chart. Plain-language template: Churn risk is moderate (50%) mainly because this customer has an order-frequency trend of -0.10/month (lowers risk); has bought 115 different products (lowers risk); has not bought for 302 days (raises risk).

LIME versus SHAP (borderline customer)



For customer 15276, LIME's top conditions were country_eire <= 0.00, freq_trend <= -0.03, spend_last_180d <= 0.00, 157.00 < recency_days <= 321.00, n_unique_products > 92.00; SHAP's top features were freq_trend, n_unique_products, recency_days, spend_last_180d, engagement_score. Both agree on: freq_trend, n_unique_products, recency_days, spend_last_180d. They diverge on the rest because LIME fits a local linear surrogate on binned features while SHAP attributes the model's actual output; for a near-50% customer the ordering of small effects is unstable in both.

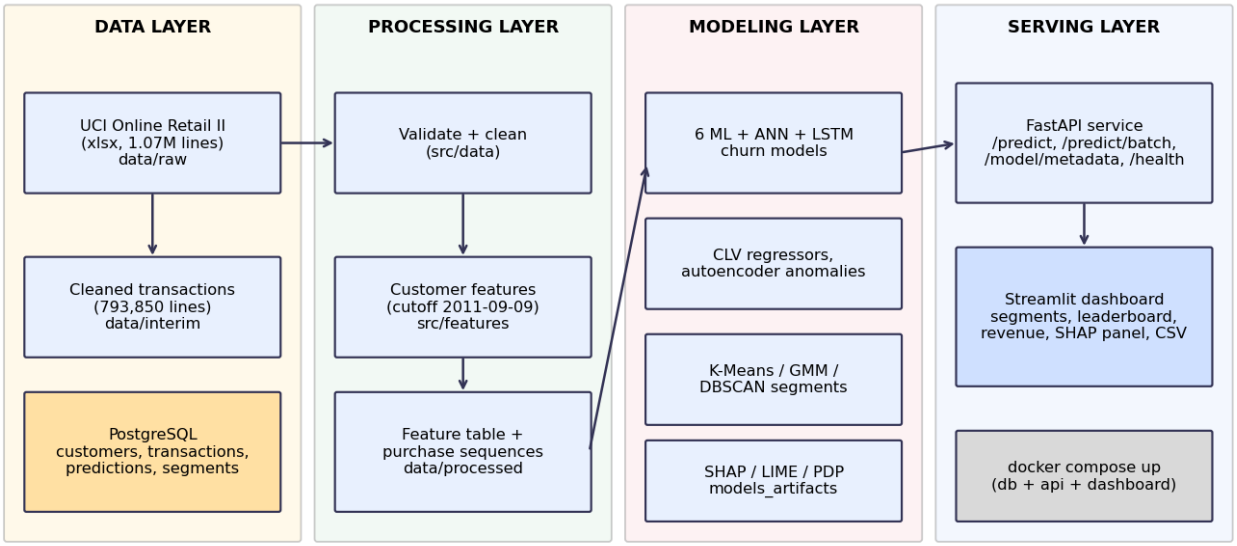


Partial dependence for the three most influential features.

8. System: API, dashboard, database and deployment

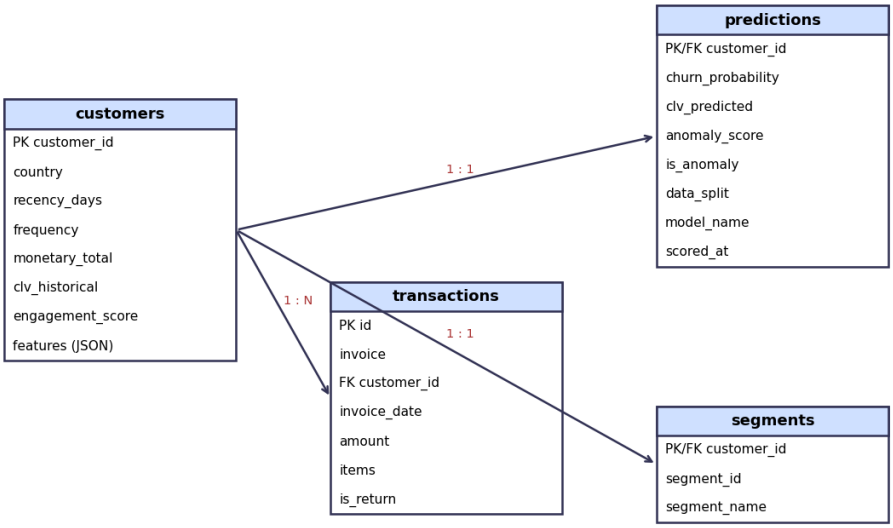
FastAPI endpoints: GET /health, GET /model/metadata, POST /predict, GET /customers/{id}/recommendations, POST /predict/batch (CSV in, JSON/CSV/PDF out), GET /customers (filters), GET /customers/leaderboard, GET /customers/{id}, GET /customers/{id}/explain, GET /segments, GET /revenue/trend. Pydantic validates inputs and returns 422 with clear messages. PostgreSQL (SQLAlchemy) stores customers, invoice-level transactions, predictions and segments; scores are precomputed so the dashboard does not rescore on page load. The Streamlit dashboard calls only the API. Everything starts with `docker compose up --build`; configuration comes from environment variables.

Vantara Customer Behavior Prediction Platform - architecture

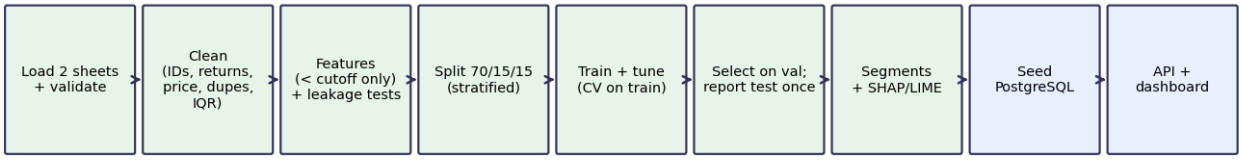


The API reads and writes PostgreSQL (predictions, segments); the dashboard only talks to the API.

PostgreSQL schema (ER diagram)



End-to-end workflow: python -m src.pipeline, then docker compose up



Every stage is scripted (no notebook-only steps) and logs to the console; config in config/config.yaml, seed 42.

9. Evaluation against the success metrics

Churn ROC-AUC 0.786 vs target 0.8 (not met). Every model, including the best on validation (xgboost, 0.811), plateaus near 0.79 on the test split, and added recent-activity features did not move it. The label is a noisy human behavior (a customer skipping three months), the data has no marketing, product or demographic signals, and there are only about 5,000 customers, so a ceiling near 0.8 is plausible; this was not investigated further with more data. **Churn recall 0.765 (met)** at the default 0.5 threshold. **CLV R2 0.415 vs target 0.6 (not met).** Next-90-day spend is heavy-tailed (maximum above 100,000 against a median of 0), 57% of customers spend nothing in the window, and a few very large customers dominate the squared error; on a log scale R2 is about 0.34. No threshold or target was tuned on the test split.

API latency: 200 sequential single-customer predictions against the running API (real PostgreSQL, localhost): p50 6.1 ms, p95 6.6 ms (includes SHAP explanation). This is a build-machine measurement on localhost, not a network or Docker measurement. **Dashboard load:** page load to both charts rendered, headless Chrome on the build machine, API on localhost

10. Limitations and honest notes

The churn label follows the brief (no purchase in the next 90 days), so it also covers customers who were already inactive for months at the cutoff (56.6% churn); the model is partly a continued-inactivity predictor, and an early-warning view limited to recently active customers is not evaluated here. k = 2 has the best silhouette; k = 3 was chosen for business use. Single time-split with one cutoff; no rolling-origin evaluation. Rows used in training are scored too and the dashboard marks which split they were in (in-sample scores look better than they are). The segmentation scaler and clusters are fitted on training customers only; the segment profile table describes all modelled customers. Product categories are a keyword proxy. The revenue chart sums invoices of modelled customers only and its forecast is a straight-line overlay, not a validated forecast. During the build the API, dashboard and tests were run against a real PostgreSQL server; afterwards the Docker Compose stack was verified end to end on Windows 11 Home (Docker Desktop 4.98.0, November 8, 2026): a fresh extraction of the delivered November 7 ZIP (this revision additionally changes the leaderboard heading text, batch-PDF number formatting, one dependency pin and documentation; the Docker-related files the run exercised are unchanged) with a newly generated .env ran docker compose up --build, all three containers came up healthy, the API seeded the database and answered requests, and the dashboard rendered all six views including batch CSV scoring and PDF export. AI tools were used to write and debug the code and this report; the author should read the code before presenting it as their own understanding.

Appendix A. Mathematical foundations

A.1 Statistics and probability

Churn is a Bernoulli outcome with prior pi = P(y=1) (here about 0.57). A classifier outputs p(x) = P(y=1 | x). The decision rule is yhat = 1 if p(x) >= t. From the confusion matrix (TP, FP, TN, FN): precision = TP/(TP+FP); recall = TP/(TP+FN); F1 = 2*precision*recall/(precision+recall); accuracy = (TP+TN)/N. ROC-AUC = P(p(x+) > p(x-)) for a random positive and negative customer, equivalently the area under TPR vs FPR as t varies; it does not depend on pi, which is why it is reported with recall and F1 instead of accuracy. Cross-validated estimates use k = 5 folds: CV = (1/k) sum_i metric(model fitted without fold i, fold i). Spend is heavy-tailed, so the CLV model fits log(1 + spend) and uses R2 = 1 - SS_res/SS_tot, which is sensitive to large errors on raw spend.

A.2 Linear algebra and PCA

With standardized feature matrix X (n x d), the covariance is C = X^T X/(n-1). PCA solves C v_j = lambda_j v_j; the projection onto the top-2 eigenvectors gives the segment plot, with explained variance lambda_j / sum(lambda). Multicollinearity (spend in 30/90/180-day windows, engagement score built from recency, frequency and spend) shows up as near-zero eigenvalues of C; the variance inflation factor VIF_j = 1/(1 - R2_j), where R2_j is the R2 of regressing feature j on the others, quantifies it. Tree models tolerate it; for logistic regression L2 regularization stabilizes it.

A.3 Logistic regression and cross-entropy

p(x) = sigmoid(w.x + b) with sigmoid(z) = 1/(1+e^-z). The loss is binary cross-entropy L = -(1/n) sum [y log p + (1-y) log(1-p)] + (lambda/2)||w||^2, with optional class weights. The gradient is dL/dw = (1/n) X^T (p - y) + lambda w, so each coefficient is a log-odds change per standardized unit and gives a first-pass explanation. XGBoost and LightGBM minimize the same loss by adding trees fitted to the gradient g_i = p_i - y_i and Hessian h_i = p_i(1-p_i) of the loss.

A.4 Gradient descent and backpropagation

Weights update as w <- w - eta * grad L(w) (Adam adds momentum and per-weight scaling; eta = 0.001 here). For a layer a_l = relu(W_l a_(l-1) + b_l), the chain rule gives delta_l = (W_(l+1)^T delta_(l+1)) * relu'(z_l) and dL/dW_l = delta_l a_(l-1)^T. The ANN stacks Linear-BatchNorm-ReLU-Dropout blocks. The LSTM carries a cell state: f_t = sigma(W_f [h_(t-1), x_t] + b_f), i_t = sigma(W_i [..] + b_i), g_t = tanh(W_g [..] + b_g), c_t = f_t * c_(t-1) + i_t * g_t, o_t = sigma(W_o [..] + b_o), h_t = o_t * tanh(c_t); gradients flow back through time via c_t. The loss curves in section 4 show training and validation loss per epoch; training stops when validation loss has not improved for 12 epochs, which is the convergence and overfitting check.

A.5 Regularization

L2 adds (lambda/2)||w||^2 (logistic regression C = 1/lambda); dropout zeroes each hidden unit with probability 0.3 during training, scaling the rest, which approximates averaging many thinner networks; early stopping restores the weights from the best validation epoch; tree depth, min_samples_leaf and subsampling limit tree variance; random forests average B trees fitted on bootstrap samples, reducing variance by roughly 1/B for uncorrelated trees. Cross-validation on the training split chooses these settings, and the test split is never used for tuning.

A.6 Clustering, explanations and anomalies

K-Means minimizes sum_k sum_(x in S_k) ||x - mu_k||^2 by alternating assignment and mean updates; silhouette s(i) = (b_i - a_i)/max(a_i, b_i) compares mean intra-cluster distance a_i with the nearest other cluster's b_i. A Gaussian mixture maximizes sum_i log sum_k pi_k N(x_i | mu_k, Sigma_k) by EM; DBSCAN groups points with at least min_samples neighbors within eps. SHAP values are Shapley values phi_j = sum over subsets S of F without j of |S|!(|F|-|S|-1)!/|F|! [f(S+j) - f(S)], with f(x) = base value + sum_j phi_j; native tree contributions (tree SHAP) compute them exactly for tree ensembles. LIME fits a weighted linear model g to f on perturbed samples around x. The autoencoder minimizes ||x - D(E(x))||^2; a customer is flagged when that error exceeds the 99th percentile on validation data.

A.7 Worked example from this project's test split (production model, threshold 0.5)

Confusion matrix: TP = 341, FP = 105, TN = 237, FN = 105 (N = 788). Precision = TP/(TP+FP) = 341/(341+105) = 0.765; recall = TP/(TP+FN) = 341/(341+105) = 0.765 (in this split FP and FN are both 105, so the two denominators coincide and precision equals recall numerically); F1 = 2 x 0.765 x 0.765 / (0.765 + 0.765) = 0.765; accuracy = (341+237)/788 = 0.734. Prior of churn in this split = 0.566, so always predicting 'churn' would give accuracy 0.566 and recall 1.0 with precision 0.566: this is why recall alone, or accuracy alone, is not enough and all of precision, recall, F1 and ROC-AUC are reported together. Mean predicted probability on the test split = 0.536 (observed rate 0.566), a quick calibration check.

A.8 Gradient boosting objective and bias-variance

A boosted model is f(x) = sum_m eta * h_m(x) with each tree h_m chosen to minimize the second-order approximation of the loss, sum_i [g_i h(x_i) + 0.5 h_i h(x_i)^2] + Omega(h), where Omega penalizes the number of leaves and leaf weights. A split is kept when the gain 0.5 [G_L^2/(H_L+lambda) + G_R^2/(H_R+lambda) - (G_L+G_R)^2/(H_L+H_R+lambda)] - gamma is positive, with G and H the sums of g_i and h_i in a node. Expected test error decomposes as bias^2 + variance + noise: shallow trees and heavy regularization raise bias; deep trees raise variance. With a noisy behavioural label the irreducible noise term is large, which is the main reason accuracy plateaus for every model family in section 4.

Appendix B. Future enhancements

Transformer sequence models (out of scope per brief, need more sequences); rolling-origin evaluation with several cutoffs; richer product taxonomy; probability calibration and a cost-based threshold using campaign cost and CLV; real-time streaming and authentication; A/B testing of retention offers; CatBoost comparison; MLflow tracking; CI with the test suite.